Scraping Kickstarter (CodeGrade)

(https://github.com/learn-co-curriculum/python-scraping-kickstarter) (https://github.com/learn-co-curriculum/python-scraping-kickstarter/issues/new)

Learning Goals

- Use BeautifulSoup to scrape an HTML document.
- Use scraped data to build a nested data structure.

Key Vocab

- **Web scraping**: Web scraping, web harvesting, or web data extraction is data scraping used for extracting data from websites
- **HTML**: The HyperText Markup Language or HTML is the standard markup language for documents designed to be displayed in a web browser.
- **CSS**: Cascading Style Sheets is a style sheet language used for describing the presentation of a document written in a markup language such as HTML or XML.
- **CSS selector**: CSS selectors define the elements to which a set of CSS rules apply.

Introduction

In this lab, you'll be scraping a Kickstarter web page that lists projects requesting funding. The page you'll be scraping displays 20 previews of projects in the NYC area. Each project has a title, an image, a short description, a location and some funding details. Our goal is to collect this information for each project and build a dictionary for each project:

```
projects = {
  "My Great Project" {
    image_link: "Image Link",
    description: "Description",
    location: "Location",
    percent_funded: "Percent Funded"
  },
  "Another Great Project" {
    image_link: "Image Link",
    description: "Description",
    location: "Location",
    percent_funded: "Percent Funded"
  }
}
```

These individual project dictionaries will be collected into a larger dictionary called `projects`.

Fixtures

In the directory of this project, you'll notice a folder called `fixtures`. Inside that folder, you'll see a file, `kickstarter.html`. If you are using the Learn IDE right click on the `kickstarter.html` file and select `Show in Finder`. Once Finder opens double click `kickstarter.html` to view the file inside your default web browser. If you are not using the Learn IDE, try open `kickstarter.html` inside your text editor and right-click anywhere on the page to select `open in browser` from the menu that appears.

Ta-da! We're looking at a web page. For the purposes of this lab, we won't be scraping a live web page. We'll be scraping this HTML page. We're doing this for two reasons. First, because web pages change. If we assign you a lab based on material that will change, things could get really confusing. Secondly, it is common to keep data that the test suite will use to test your program in a `fixtures` directory.

So, for this lab, we *don't need requests*. We're not opening a live web page.

Instructions

Setting Up Our Project

Since we'll be using that `kickstarter.html` file instead of an requests request, we need to require only `Beautiful Soup` at the top of the `kickstarter_scraper.py` file

Next, let's set up some variables inside the method called `create_project_dict` :

```
html = ''
with open('./fixtures/kickstarter.html') as file:
    html = file.read()

kickstarter = BeautifulSoup(html, 'html.parser')
```

Notice that this is pretty similar to what we did to open HTML documents in the previous exercise in which we did use requests.

Selecting the Projects

The first thing we'll want to do is figure out what selector will allow us to grab each project as a whole. Open up `fixtures/kickstarter.html` by typing:

```
open fixtures/kickstarter.html
```

in the terminal, or by right clicking on the file and selecting "open in browser".

This should open the file in your web browser. Right click somewhere on the "Moby Dick" project and choose "Inspect Element". By moving your mouse up and down in the HTML in the inspector, you can see what each element represents on the page via some cool highlighting. By moving your mouse around, it quickly becomes clear that each project is contained in:

```
<li class="project grid_4">...</li>
```

Since this BeautifulSoup object is just a bunch of nested nodes, and we know how to iterate through a nested data structure, we can use the Python we already know to iterate through each of these projects and do stuff with them.

Just to check our assumptions, let's add a `import ipdb` at the top of our file, and add `ipdb.set_trace()` after the last line. Add a call to the `create_project_dict` method at the bottom of the file.

```
from bs4 import BeautifulSoup
import ipdb

html = ''
def create_project_dict():

    with open('./fixtures/kickstarter.html') as file:
        html = file.read()

    kickstarter = BeautifulSoup(html, 'html.parser')
    ipdb.set_trace()

create_project_dict()
```

Then type `python kickstarter_scraper.py` into your terminal. This should drop us into ipdb, so that we can play around.

In ipdb, type in:

```
kickstarter.select("li.project.grid_4")[0]
```

This will select the first `li` with the `project` and `grid_4` classes just so that we can make sure we've chosen our selectors correctly.

And we have! (If you don't see any output, or see an empty list, make sure you've typed everything exactly as it was typed here.)

Awesome! Let's add a comment to `kickstarter_scraper.py` that reminds us of that selector:

```
# projects: kickstarter.select("li.project.grid_4")[0]
```

Selecting the Title

Let's hop back into ipdb and see if we can figure out how to get the title of that project.

In ipdb, type:

```
project = kickstarter.select("li.project.grid_4")[0]
```

This will assign that project to a variable, `project` so that we can play around with it.

Go back to your browser and use the element inspector to click around a bit and identify the selector for a project's title. A bit of inspection should reveal that the title of each project lives in an `h2` with a class of `bbcard_name`, inside a `strong` and then an `a` tag. Let's check that in ipdb:

```
project.select("h2.bbcard_name strong a")[0].text
```

Since BeautifulSoup gives us a bunch of nested nodes that all respond to the same methods, we can just chain a `select` method right onto this `project`. Neat, huh?

Now that we have our `title` selector, let's add it into a comment in our `kickstarter_scraper.py`.

```
# projects: kickstarter.select("li.project.grid_4")[0]
# title: project.select("h2.bbcard_name strong a")[0].text
```

Selecting the Image Link

Back in Chrome, we can see in the inspector that there is a `div` with a class of `project-thumbnail`. Seems like a good place to look. Let's give it a try in ipdb.

In ipdb, type:

```
project.select("div.project-thumbnail a img")[0]['src']
```

It worked! Now, let's continue to keep track of our working code in our project file:

```
# projects: kickstarter.select("li.project.grid_4")[0]
# title: project.select("h2.bbcard_name strong a")[0].text
# image link: project.select("div.project-thumbnail a img")[0]['src']
```

A Note on Attributes

A tag may have any number of attributes. The tag `<img src="http://www.example.com/pic.jpg">` has an attribute src whose value is '**http://www.example.com/pic.jpg (http://www.example.com/pic.jpg)**'. You can access a tag's attributes by treating the tag like a dictionary:

```
tag['src']
```

Selecting the Description

Are you starting to see a pattern here? We click around a bit in the Chrome web inspector, take a stab at a CSS selector in ipdb, and then keep track of that selector in our project file. Let's grab the description now. In ipdb:

```
project.select("p.bbcard_blurb")[0].text
```

This should return the description of an individual project.

Let's add that to `kickstarter_scraper.py` :

```
# projects: kickstarter.select("li.project.grid_4")[0]
# title: project.select("h2.bbcard_name strong a")[0].text
# image link: project.select("div.project-thumbnail a img")[0]['src']
# description: project.select("p.bbcard_blurb")[0].text
```

Selecting the Location

Do you think you can figure this one out on your own? Examine the web page and then play around in ipdb. Try to find the right selector for an individual project's location.

Selecting the Percent Funded

And last, but not least, let's try and grab the percent funded as well! Looking in Chrome, it seems that this one is just a bit trickier, but only because it's more nested than the other ones. In ipdb, type:

```
project.select("ul.project-stats li.first.funded strong")[0].text
```

That does it! To make it useful for later on if, say, we wanted to do some math, let's also tag on a `.replace("%", "")` to remove the percent sign.

Our final list of comments in our `kickstarter_scraper.py` file, then (including the location that you should have figured out on your own), is:

```
# projects: kickstarter.select("li.project.grid_4")[0]
# title: project.select("h2.bbcard_name strong a")[0].text
# image link: project.select("div.project-thumbnail a img")[0]['src']
# description: project.select("p.bbcard_blurb")[0].text
# location: project.select("ul.project-meta span.location-name")[0].text
# percent_funded: project.select("ul.project-stats li.first.funded strong")[0].text.replace("%","")
```

Let's Scrape

Now, it's just a matter of putting together the data we can grab with BeautifulSoup with our knowledge of data iteration in Python.

First, let's set up a loop to iterate through the projects (and also an empty `projects` dictionary, which we will fill up with scraped data):

```
# file: kickstarter_scraper.py
from bs4 import BeautifulSoup
import ipdb
# projects: kickstarter.select("li.project.grid_4")[0]
# title: project.select("h2.bbcard_name strong a")[0].text
```

```
# image link: project.select("div.project-thumbnail a img")[0]['src']
# description: project.select("p.bbcard_blurb")[0].text
# location: project.select("ul.project-meta span.location-name")[0].text
# percent_funded: project.select("ul.project-stats li.first.funded strong")[0].text.replace("%","")
def create_project_dict():
  html = ''
  with open('./fixtures/kickstarter.html') as file:
        html = file.read()
  kickstarter = BeautifulSoup(html, 'html.parser')
  projects = {}
  # Iterate through the projects
  for project in kickstarter.select("li.project.grid_4"):
    projects[project] = {}
  # return the projects dictionary
  return projects
```

Ok, so that won't work, actually. That's going to make some really wacky key which is a huge BeautifulSoup object. So, let's change our data structure slightly and make it so that each project title is a key, and the value is another dictionary with each of our other data points as keys. Sound good?

```
# file: kickstarter_scraper.py
...
def create_project_dict():
  projects = {}
  # Iterate through the projects
  for project in kickstarter.select("li.project.grid_4"):
    projects[title] = {}
  # return the projects dictionary
  return projects
```

That's better.

Finally, it's just a matter of grabbing each of the data points using the selectors we've already figured out, and adding them to each project's dictionary. So, our complete code will look something like this:

Solution Code

```
# file: kickstarter_scraper.py
from bs4 import BeautifulSoup
import ipdb

# projects: kickstarter.select("li.project.grid_4")[0]
# title: project.select("h2.bbcard_name strong a")[0].text
# image link: project.select("div.project-thumbnail a img")[0]['src']
# description: project.select("p.bbcard_blurb")[0].text
# location: project.select("ul.project-meta span.location-name")[0].text
# percent_funded: project.select("ul.project-stats li.first.funded strong")[0].text.replace("%","")

def create_project_dict():
  html = ''
  with open('./fixtures/kickstarter.html') as file:
        html = file.read()

  kickstarter = BeautifulSoup(html, 'html.parser')
  projects = {}
  # Iterate through the projects
  for project in kickstarter.select("li.project.grid_4"):
    title = project.select("h2.bbcard_name strong a")[].text
    projects[title] = {
      'image_link': project.select("div.project-thumbnail a img").attribute("src").value,
      'description': project.select("p.bbcard_blurb")[0].text,
      'location': project.select("ul.project-meta span.location-name")[0].text,
      'percent_funded': project.select("ul.project-stats li.first.funded strong")[0].text.replace("%","")

  # return the projects dictionary
```

```
    return projects
```

This tool needs to be loaded in a new browser window

Load Scraping Kickstarter (CodeGrade) in a new window